

Td-Tp 7 : Optimisation des hyperparamètres

Cette fois, ces exercices prennent comme exemple le réseau neuronal entièrement connecté dans Td-Tp 3 pour optimiser les hyperparamètres et éviter le surapprentissage pendant le processus de formation du réseau.

Ici, nous mettons le code que tout le monde a complété dans le quatrième Tp directement devant pour votre référence ultérieure.

```
In [2]: import numpy as np
import copy
import matplotlib.pyplot as plt
import h5py
import scipy
from PIL import Image
from scipy import ndimage

%matplotlib inline
%load_ext autoreload
%autoreload 2

def load_dataset():
    train_dataset = h5py.File('datasets/train_catvnoncat.h5', "r")
    train_set_x_orig = np.array(train_dataset["train_set_x"][:]) # your train set features
    train_set_y_orig = np.array(train_dataset["train_set_y"][:]) # your train set labels

    test_dataset = h5py.File('datasets/test_catvnoncat.h5', "r")
    test_set_x_orig = np.array(test_dataset["test_set_x"][:]) # your test set features
    test_set_y_orig = np.array(test_dataset["test_set_y"][:]) # your test set labels

    classes = np.array(test_dataset["list_classes"][:]) # the list of classes

    train_set_y_orig = train_set_y_orig.reshape((1, train_set_y_orig.shape[0]))
    test_set_y_orig = test_set_y_orig.reshape((1, test_set_y_orig.shape[0]))

    return train_set_x_orig, train_set_y_orig, test_set_x_orig, test_set_y_orig, classes

train_set_x_orig, train_set_y, test_set_x_orig, test_set_y, classes = load_dataset()

n_train = train_set_x_orig.shape[0]
n_test = test_set_x_orig.shape[0]
num_px = train_set_x_orig.shape[1]

train_set_x_flatten = train_set_x_orig.reshape(train_set_x_orig.shape[0], -1).T
test_set_x_flatten = test_set_x_orig.reshape(test_set_x_orig.shape[0], -1).T

train_set_x = train_set_x_flatten / 255.
test_set_x = test_set_x_flatten / 255.
```

```

In [3]: def sigmoid(z):
        return 1/(1+np.exp(-z))

def initialize_with_zeros(dim):
    w = np.zeros([dim,1])
    b = 0.0
    return w, b

def propagate(w, b, X, Y):
    n = X.shape[1]

    ## PROPAGATION AVANT (DE X AU COÛT)
    hat_Y = sigmoid(np.dot(w.T,X)+b)
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T))

    ## PROPAGATION ARRIÈRE (POUR TROUVER LE GRAD)
    dw = 1/n * np.dot(X, (hat_Y-Y).T)
    db = 1/n * np.sum(hat_Y-Y)

    cost = np.squeeze(np.array(cost))

    grads = {"dw": dw,
             "db": db}
    return grads, cost

def optimize(w, b, X, Y, num_iterations=100, learning_rate=0.009, print_cost=False):
    w = copy.deepcopy(w)
    b = copy.deepcopy(b)

    costs = []

    for i in range(num_iterations):
        ## Calcul du coût et du gradient
        grads, cost = propagate(w, b, X, Y)

        ## Récupérer les dérivés de grads
        dw = grads["dw"]
        db = grads["db"]

        ## mise à jour
        w += -learning_rate * dw
        b += -learning_rate * db

        ## Enregistrer les coûts
        if i % 100 == 0:
            costs.append(cost)

            ## Imprimer le coût toutes les 100 itérations d'entraînement
            if print_cost:
                print ("Coût après itération %i: %f" %(i, cost))

    params = {"w": w,
             "b": b}

    grads = {"dw": dw,
             "db": db}
    return params, grads, costs

def predict(w, b, X):
    n = X.shape[1]
    Y_prediction = np.zeros((1, n))
    w = w.reshape(X.shape[0], 1)

    ## Calculer le vecteur "hat_Y" prédisant les probabilités qu'un chat soit présent dans l'i
    hat_Y = sigmoid(np.dot(w.T,X)+b)

    for i in range(hat_Y.shape[1]):
        ## Convertir les probabilités hat_Y[0,i] en prédictions réelles p[0,i]
        if hat_Y[0, i] > 0.5 :
            Y_prediction[0,i] = 1
        else:
            Y_prediction[0,i] = 0
    return Y_prediction

def model(X_train, Y_train, X_test, Y_test, num_iterations=2000, learning_rate=0.5, print_cost:
    w, b = initialize_with_zeros(X_train.shape[0])

```

```

## Descente de gradient
params, grads, costs = optimize(w, b, X_train, Y_train, num_iterations, learning_rate, pri

## Récupérer les paramètres w et b du dictionnaire `params`
w = params["w"]
b = params["b"]

## Prédire les exemples d'ensembles de test/d'entraînement
Y_prediction_test = predict(w, b, X_test)
Y_prediction_train = predict(w, b, X_train)

## Imprimer les erreurs d'entraînement/de test
# pe et pt sont actuellement les exactitudes des entraînements et des tests (selon l'expli
# précision (exacte), rappel, f1_score sont aussi applicable
pe = 100 - np.mean(np.abs(Y_prediction_train - Y_train)) * 100
pt = 100 - np.mean(np.abs(Y_prediction_test - Y_test)) * 100
if print_cost:
    print("précision des entraînements : {} %".format(pe))
    print("précision des tests : {} %".format(pt))

d = {"costs": costs,
     "Y_prediction_test": Y_prediction_test,
     "Y_prediction_train": Y_prediction_train,
     "w": w,
     "b": b,
     "learning_rate": learning_rate,
     "num_iterations": num_iterations,
     "res": np.sign(pe-pt)*pt}
return d

```

Matrice de confusion

Une fois que vous avez construit un modèle de classification, vous devez évaluer la qualité des prédictions faites par ce modèle. Alors, comment définissez-vous les « bonnes » prédictions ?

Certaines mesures de performance nous aident à améliorer nos modèles. Explorons les différences entre elles pour un problème de classification binaire :

Considérez la matrice de confusion suivante pour un problème de classification :

		Predicted	
		Yes	No
Actual	Yes	True Positive (TP) = 20	False Negative (FN) = 10
	No	False Positive (FP) = 5	True Negative (TN) = 65

Voici maintenant les mesures fondamentales pour les données ci-dessus :

1. **Précision (Precision)** : Il s'agit de la mesure des cas positifs correctement identifiés parmi tous les cas positifs prédits.

Elle est utile lorsque les coûts des faux positifs sont élevés.

$$precision = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Positives\ (FP)} = \frac{20}{20 + 5} = 80\%$$

2. **Rappel (Recall)** : C'est la mesure des cas positifs correctement identifiés parmi l'ensemble des cas positifs réels. Elle est importante lorsque le coût des faux négatifs est élevé.

$$recall = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Negatives\ (FN)} = \frac{20}{20 + 10} \approx 66.6\%$$

3. **Exactitude (Accuracy)** : L'une des mesures les plus évidentes, c'est la mesure de tous les cas correctement identifiés.

Elle est surtout utilisée lorsque toutes les classes sont d'égale importance. Elle est surtout utilisée lorsque toutes les classes sont d'égale importance.

$$accuracy = \frac{True\ Positives\ (TP) + True\ Negatives\ (TN)}{True\ Positives\ (TP) + False\ Positives\ (FP) + True\ Negatives\ (TN) + False\ Negatives\ (FN)} =$$

Fonction d'objectif d'optimisation personnalisée (Par exemple, précision, score F1, etc.)

1. [accuracy_score](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html) (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html)

`accuracy_score` est une fonction de la bibliothèque Scikit-learn qui calcule la précision d'un modèle de classification. Si $\hat{y}_i$ est la valeur prédite du i -ème échantillon et y_i est la vraie valeur correspondante, alors la fraction de prédictions correctes sur $n_{samples}$ est définie comme

$$accuracy(y, \hat{y}) = \frac{1}{n_{samples}} \sum_{i=0}^{n_{samples}-1} \mathbb{1}(\hat{y}_i = y_i)$$

où $\mathbb{1}(x)$ est la fonction d'indicateur.

```
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_true, y_pred)
```

2. [precision_score](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html) (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html) La précision (precision) fait référence à la proportion d'échantillons prédits comme positifs par le modèle qui le sont réellement :

$$precision = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Positives\ (FP)}$$

```
from sklearn.metrics import precision_score
precision = precision_score(y_true, y_pred)
```

3. [recall_score](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html) (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html) Le rappel (recall) fait référence à la proportion d'échantillons réellement positifs qui sont correctement prédits comme positifs par le modèle :

$$recall = \frac{True\ Positives\ (TP)}{True\ Positives\ (TP) + False\ Negatives\ (FN)}$$

```
from sklearn.metrics import recall_score
recall = recall_score(y_true, y_pred)
```

4. [f1_score](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score) (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score)

`f1_score` est un indicateur utilisé pour évaluer les performances des modèles de classification, particulièrement adapté aux ensembles de données déséquilibrés. Sa formule de calcul est :

$$F1 - score = 2 \times \frac{precision + recall}{precision \times recall}$$

Ici,

```
from sklearn.metrics import f1_score
f1 = f1_score(y_true, y_pred)
```

5. [roc_auc_score](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html) (https://scikit-learn.org/stable/modules/generated/sklearn.metrics.roc_auc_score.html)

```
from sklearn.metrics import roc_auc_score
```

Table des matières

- [I - Réalisation sans l'aide de Scikit Learn](#)
 - [1.1 - Optimisation](#)
 - [1.1.1 - Recherche par grille](#)
 - [1.1.2 - Recherche aléatoire](#)
 - [Exercice 1 : Recherche aléatoire](#)
 - [1.2 - Surapprentissage](#)
 - [1.2.1 - LASSO](#)
 - [1.2.2 - Ridge](#)
 - [Exercice 2 : Ridge](#)
 - [1.2.3 - Recherche par grille avec validation croisée](#)
 - [1.2.4 - Recherche aléatoire avec validation croisée](#)
 - [Exercice 3 : Recherche aléatoire avec validation croisée](#)
- [II - Réalisation à l'aide de Scikit Learn](#)
 - [2.1 - Optimisation](#)
 - [2.1.1 - Recherche par grille](#)
 - [2.1.2 - Recherche aléatoire](#)
 - [2.1.3 - Optimisation bayésienne](#)
 - [2.2 - Surapprentissage](#)
 - [2.2.1 - LASSO et Ridge](#)
- [III - Réseaux Adverses Génératifs \(GAN\)](#)
 - [3.1 - Générateur](#)
 - [Exercice 4 : Generator](#)
 - [3.2 - Discriminateur](#)
 - [Exercice 5 : Discriminator](#)
 - [3.3 - Boucle d'entraînement du GAN](#)
 - [Exercice 6 : train_gan](#)

I - Réalisation sans l'aide de Scikit Learn

1.1 - Optimisation

1.1.1 - Recherche par grille

Les combinaisons qui doivent être évaluées par l'utilisateur sont testées avec `GridSearchCV` dans la bibliothèque `Sklearn`. En fait, le modèle s'adapte à chaque combinaison individuellement, révélant les meilleurs résultats et paramètres. Par exemple, lorsque nous considérons `LogisticRegression`, si 4 valeurs différentes sont sélectionnées pour taux d'apprentissage et 4 valeurs différentes sont sélectionnées pour le nombre total d'itération, le modèle s'ajustera 16 fois et le résultat de chacune sera représenté. Créons maintenant une recherche par grille sur l'ensemble de données sans utiliser la bibliothèque `sklearn` :

In [4]: `## Recherche par grille`

```
best_lr = 0
for lr in [0.001, 0.005, 0.01, 0.05]:
    for ni in [500, 1000, 2000, 3000]:
        logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat
        lr_score = logistic_regression_model["res"]
        print("num_iterations: ", ni, "learning_rate: ", lr, 'res: {:.3f}'.format(lr_score))
        if lr_score > best_lr:
            best_lr = lr_score
            best_lr_combination = (ni, lr)
print("best score LogisticRegression",best_lr)
print("best num_iterations and learning_rate",best_lr_combination)
```

```
num_iterations: 500 learning_rate: 0.001 res: 56.000
num_iterations: 1000 learning_rate: 0.001 res: 62.000
num_iterations: 2000 learning_rate: 0.001 res: 68.000
num_iterations: 3000 learning_rate: 0.001 res: 74.000
num_iterations: 500 learning_rate: 0.005 res: 74.000
num_iterations: 1000 learning_rate: 0.005 res: 72.000
num_iterations: 2000 learning_rate: 0.005 res: 70.000
num_iterations: 3000 learning_rate: 0.005 res: 68.000
num_iterations: 500 learning_rate: 0.01 res: 82.000
num_iterations: 1000 learning_rate: 0.01 res: 70.000
num_iterations: 2000 learning_rate: 0.01 res: 70.000
num_iterations: 3000 learning_rate: 0.01 res: 68.000
num_iterations: 500 learning_rate: 0.05 res: -78.000
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/2332453114.py:15: RuntimeWarning:
divide by zero encountered in log
cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T))
```

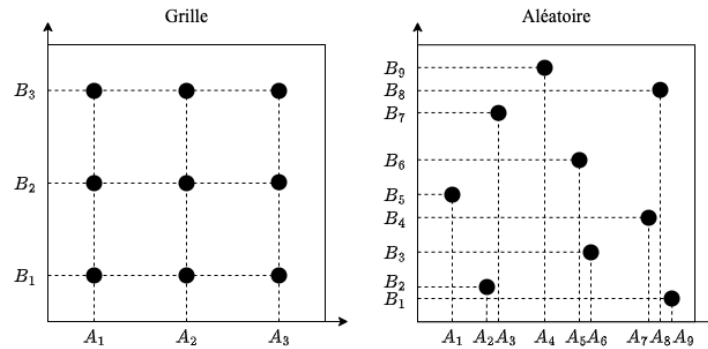
```
num_iterations: 1000 learning_rate: 0.05 res: 70.000
num_iterations: 2000 learning_rate: 0.05 res: 68.000
num_iterations: 3000 learning_rate: 0.05 res: 66.000
best score LogisticRegression 82.0
best num_iterations and learning_rate (500, 0.01)
```

In [5]: `logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat`

```
Coût après itération 0: 0.693147
Coût après itération 100: 0.823921
Coût après itération 200: 0.418944
Coût après itération 300: 0.617350
Coût après itération 400: 0.522116
précision des entraînements : 83.25358851674642 %
précision des tests : 82.0 %
```

1.1.2 - Recherche aléatoire

Par rapport à l'algorithme de recherche par grille précédent, dans l'algorithme de recherche aléatoire, nous prenons le même nombre de groupes de paramètres, mais les groupes de paramètres ici sont des nombres aléatoires dans un intervalle donné. Veuillez utiliser `np.random.seed(1)`, `np.random.rand` et `np.random.randint` pour compléter cette partie du code.



Exercice 1 : Recherche aléatoire

```
In [31]: ## Recherche aléatoire
# 16 points vérifiés
# 0.001 <= lr < 0.05
# 500 <= ni < 3000

np.random.seed(1)
best_lr = 0
for iter in range(16):
    lr = 0.001 + (0.05-0.001) * np.random.rand()
    ni = np.random.randint(500,3000)
    logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iter=ni)
    lr_score = logistic_regression_model["res"]
    print("num_iterations: ", ni, "learning_rate: ", lr, 'res: {:.3f}'.format(lr_score))
    if lr_score > best_lr:
        best_lr = lr_score
        best_lr_combination = (ni, lr)
print("best score LogisticRegression",best_lr)
print("best num_iterations and learning_rate",best_lr_combination)
```

```
num_iterations: 1596 learning_rate: 0.02143407823042613 res: 68.000
num_iterations: 1460 learning_rate: 0.0010056043660498994 res: 64.000
num_iterations: 1702 learning_rate: 0.005524591143671092 res: 70.000
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning:
divide by zero encountered in log
cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.
sum(w**2)
```

```
num_iterations: 857 learning_rate: 0.04246923491761484 res: 70.000
num_iterations: 752 learning_rate: 0.03457575551944122 res: 68.000
num_iterations: 2946 learning_rate: 0.04402775438315633 res: 66.000
num_iterations: 1253 learning_rate: 0.03385290799874171 res: 66.000
num_iterations: 2388 learning_rate: 0.007878959991166456 res: 70.000
num_iterations: 2525 learning_rate: 0.03914107258049014 res: 66.000
num_iterations: 819 learning_rate: 0.048444817210250485 res: 70.000
num_iterations: 1837 learning_rate: 0.005547239623396158 res: 70.000
num_iterations: 816 learning_rate: 0.04394306846250588 res: 70.000
num_iterations: 1228 learning_rate: 0.04162819846098994 res: 66.000
num_iterations: 931 learning_rate: 0.002913684378411236 res: 74.000
num_iterations: 1042 learning_rate: 0.00390291686377661 res: 74.000
num_iterations: 1978 learning_rate: 0.030060210396604788 res: 68.000
best score LogisticRegression 74.0
best num_iterations and learning_rate (931, 0.002913684378411236)
```

Output attendu :

```
best score LogisticRegression 74.0
best num_iterations and learning_rate (931, 0.002913684378411236)
```

1.2 - Surapprentissage

1.2.1 - LASSO

- L'implémentation de LASSO consiste à ajouter un terme de régularisation L1 à la fonction de coût. Ce terme de régularisation est le produit de la somme des valeurs absolues du vecteur de poids w et d'un paramètre d'ajustement λ . Plus λ est grand, plus l'effet de la régularisation est fort, ce qui peut rendre le modèle plus parcimonieux, c'est-à-dire que les coefficients des caractéristiques tendent vers zéro.

$$J = L + s = L + \frac{\lambda}{n} \sum_i |w_i|$$

avec $L = -\frac{1}{n} (y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y}))$.

- Une caractéristique distinctive de LASSO est sa capacité à effectuer une sélection de caractéristiques, c'est-à-dire qu'il peut réduire à zéro les coefficients des caractéristiques moins importantes, simplifiant ainsi le modèle et améliorant son pouvoir explicatif.

```

In [7]: import numpy as np
import copy

def sigmoid(z):
    return 1/(1+np.exp(-z))

def initialize_with_zeros(dim):
    w = np.zeros([dim,1])
    b = 0.0
    return w, b

def propagate(w, b, X, Y, lambd):
    n = X.shape[1]

    ## PROPAGATION AVANT (DE X AU COÛT)
    hat_Y = sigmoid(np.dot(w.T,X)+b)
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np

    ## PROPAGATION ARRIÈRE (POUR TROUVER LE GRAD)
    dw = 1/n * np.dot(X,(hat_Y-Y).T) + (lambd / n) * np.sign(w)
    db = 1/n * np.sum(hat_Y-Y)

    cost = np.squeeze(np.array(cost))

    grads = {"dw": dw,
             "db": db}
    return grads, cost

def optimize(w, b, X, Y, num_iterations=100, learning_rate=0.009, lambd=0.1, print_cost=False)
w = copy.deepcopy(w)
b = copy.deepcopy(b)

costs = []

for i in range(num_iterations):
    ## Calculer du coût et du gradient
    grads, cost = propagate(w, b, X, Y, lambd)

    ## Récupérer les dérivés de grads
    dw = grads["dw"]
    db = grads["db"]

    ## Mettre à jour
    w += -learning_rate * dw
    b += -learning_rate * db

    ## Sauvegarder les coûts
    if i % 100 == 0:
        costs.append(cost)

    ## Imprimer le coût chaque 100 itérations
    if print_cost:
        print ("Cost after iteration %i: %f" %(i, cost))

params = {"w": w,
          "b": b}

grads = {"dw": dw,
          "db": db}
return params, grads, costs

def model(X_train, Y_train, X_test, Y_test, num_iterations=2000, learning_rate=0.5, lambd=0.1,
w, b = initialize_with_zeros(X_train.shape[0])

## Descente de gradient
params, grads, costs = optimize(w, b, X_train, Y_train, num_iterations, learning_rate, laml

## Récupérer les paramètres w et b du dictionnaire `params`
w = params["w"]
b = params["b"]

## Prédire les exemples d'ensembles de test/d'entraînement
Y_prediction_test = predict(w, b, X_test)
Y_prediction_train = predict(w, b, X_train)

## Imprimer les erreurs d'entraînement / de test
train_accuracy = 100 - np.mean(np.abs(Y_prediction_train - Y_train)) * 100
test_accuracy = 100 - np.mean(np.abs(Y_prediction_test - Y_test)) * 100
if print_cost:
    print("Training accuracy: {} %".format(train_accuracy))
    print("Test accuracy: {} %".format(test_accuracy))

```

```

d = {"costs": costs,
     "Y_prediction_test": Y_prediction_test,
     "Y_prediction_train" : Y_prediction_train,
     "w" : w,
     "b" : b,
     "learning_rate" : learning_rate,
     "num_iterations": num_iterations,
     "train_accuracy" : train_accuracy,
     "test_accuracy" : test_accuracy,
     "res" : np.sign(train_accuracy-test_accuracy)*test_accuracy}
return d

```

```

In [8]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y,
                                         num_iterations=1000,
                                         learning_rate=0.01,
                                         lambda = 0.2,
                                         print_cost=True)

```

```

Cost after iteration 0: 0.693147
Cost after iteration 100: 11.444507
Cost after iteration 200: 14.124843
Cost after iteration 300: 18.228374
Cost after iteration 400: 21.748873
Cost after iteration 500: 24.726451
Cost after iteration 600: 26.954070
Cost after iteration 700: 28.519112
Cost after iteration 800: 29.527188
Cost after iteration 900: 30.125343
Training accuracy: 91.38755980861244 %
Test accuracy: 76.0 %

```

```

In [9]: best_score = 0
for l in [0.01, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0]:
    logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y,
                                      num_iterations=1000,
                                      learning_rate=0.01,
                                      lambda = l,
                                      print_cost=False)

    lr_score = logistic_regression_model["res"]
    print("lambda: ", l, 'res: {:.3f}'.format(lr_score))
    if lr_score > best_score:
        best_score = lr_score
        best_lambda = l
print("best score LogisticRegression",best_score)
print("best lambda",best_lambda)

```

```

lambda: 0.01 res: 70.000
lambda: 0.05 res: 72.000
lambda: 0.1 res: 70.000
lambda: 0.2 res: 76.000
lambda: 0.3 res: 80.000
lambda: 0.5 res: 34.000
lambda: 1.0 res: 40.000
best score LogisticRegression 80.0
best lambda 0.3

```

1.2.2 - Ridge

- Ridge est implémenté en ajoutant un terme de régularisation L2 à la fonction de coût. Ce terme de régularisation est le produit de la somme des carrés du vecteur de poids w et d'un paramètre d'ajustement λ . Par rapport à la régularisation L1, la régularisation L2 a tendance à rendre les coefficients de poids proches de zéro mais pas égaux à zéro, elle ne fait donc pas disparaître complètement le coefficient de caractéristique.

$$J = L + s = L + \frac{\lambda}{n} \sum_i w_i^2$$

- La caractéristique principale de Ridge est qu'elle peut réduire la corrélation entre les caractéristiques et qu'elle est insensible aux valeurs aberrantes.

Exercice 2 : Régularisation L2

Complétez cette partie du code en vous basant sur le code ci-dessus.

```

In [10]: ## Régularisation L2
# l in [0.01, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0]
# dw = 1/n * np.dot(X, (hat_Y-Y).T) + (lambda / n) * 2 * w

def propagate(w, b, X, Y, lambda):
    n = X.shape[1]

    ## PROPAGATION AVANT (DE X AU COÛT)
    hat_Y = sigmoid(np.dot(w.T,X)+b)
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambda * np

    ## PROPAGATION ARRIÈRE (POUR TROUVER LE GRAD)
    dw = 1/n * np.dot(X, (hat_Y-Y).T) + (lambda / n) * 2 * w
    db = 1/n * np.sum(hat_Y-Y)

    cost = np.squeeze(np.array(cost))

    grads = {"dw": dw,
             "db": db}
    return grads, cost

def optimize(w, b, X, Y, num_iterations=100, learning_rate=0.009, lambda=0.1, print_cost=False)
w = copy.deepcopy(w)
b = copy.deepcopy(b)

costs = []

for i in range(num_iterations):
    ## Calculer du coût et du gradient
    grads, cost = propagate(w, b, X, Y, lambda)

    ## Récupérer les dérivés de grads
    dw = grads["dw"]
    db = grads["db"]

    ## Mettre à jour
    w += -learning_rate * dw
    b += -learning_rate * db

    ## Sauvegarder les coûts
    if i % 100 == 0:
        costs.append(cost)

        ## Imprimer le coût chaque 100 itérations
        if print_cost:
            print ("Cost after iteration %i: %f" %(i, cost))

params = {"w": w,
          "b": b}

grads = {"dw": dw,
          "db": db}
return params, grads, costs

def model(X_train, Y_train, X_test, Y_test, num_iterations=2000, learning_rate=0.5, lambda=0.1,
w, b = initialize_with_zeros(X_train.shape[0])

## Descente de gradient
params, grads, costs = optimize(w, b, X_train, Y_train, num_iterations, learning_rate, lambda)

## Récupérer les paramètres w et b du dictionnaire `params`
w = params["w"]
b = params["b"]

## Prédire les exemples d'ensembles de test/d'entraînement
Y_prediction_test = predict(w, b, X_test)
Y_prediction_train = predict(w, b, X_train)

## Imprimer les erreurs d'entraînement / de test
train_accuracy = 100 - np.mean(np.abs(Y_prediction_train - Y_train)) * 100
test_accuracy = 100 - np.mean(np.abs(Y_prediction_test - Y_test)) * 100
if print_cost:
    print("Training accuracy: {} %".format(train_accuracy))
    print("Test accuracy: {} %".format(test_accuracy))

d = {"costs": costs,
     "Y_prediction_test": Y_prediction_test,
     "Y_prediction_train": Y_prediction_train,
     "w": w,
     "b": b,
     "learning_rate": learning_rate,

```

```

        "num_iterations": num_iterations,
        "train_accuracy" : train_accuracy,
        "test_accuracy" : test_accuracy,
        "res" : np.sign(train_accuracy-test_accuracy)*test_accuracy}
    return d

best_score = 0
for l in [0.01, 0.05, 0.1, 0.2, 0.3, 0.5, 1.0]:
    logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y,
                                     num_iterations=1000,
                                     learning_rate=0.01,
                                     lambd = l,
                                     print_cost=False)
    lr_score = logistic_regression_model["res"]
    print("lambda: ", l, 'res: {:.3f}'.format(lr_score))
    if lr_score > best_score:
        best_score = lr_score
        best_lambda = l
print("best score LogisticRegression",best_score)
print("best lambda",best_lambda)

```

```

lambda: 0.01 res: 70.000
lambda: 0.05 res: 70.000
lambda: 0.1 res: 70.000
lambda: 0.2 res: 70.000
lambda: 0.3 res: 72.000
lambda: 0.5 res: 72.000
lambda: 1.0 res: 72.000
best score LogisticRegression 72.0
best lambda 0.3

```

Output attendu :

```

best score LogisticRegression 72.0
best lambda 0.3

```

1.2.3 - Recherche par grille avec validation croisée

L'ensemble de données est divisé en un ensemble d'entraînement et un ensemble de test. La séparation de l'ensemble de données d'entraînement en ensemble de formation et ensemble de validation se fait par validation croisée. Implémentons-le sans utiliser la bibliothèque sklearn pour comprendre le système :

```
In [11]: best_score=0
for lr in [0.001, 0.005, 0.01, 0.05]:
    for ni in [500, 1000, 2000, 3000]:
        cv_scores = []
        for cv in range(5):
            X = train_set_x
            y = train_set_y
            logistic_regression_model = model(np.hstack((X[:, :(cv*40)], X[:, (cv*40)+40:])), y,
                                              X[:, (cv*40):(cv*40+40)], y[:, (cv*40):(cv*40+40)])
            cv_score = logistic_regression_model["res"]
            cv_scores.append(cv_score)
        mean_score=np.mean(cv_scores)
        print("num_iterations: ", ni, "learning_rate: ", lr, 'res: {:.3f}'.format(mean_score))
        if mean_score > best_score:
            best_score = mean_score
            best_score_combination = (ni, lr)
print("best score LogisticRegression",best_score)
print("best num_iterations and learning_rate",best_score_combination)
```

```
num_iterations: 500 learning_rate: 0.001 res: 59.500
num_iterations: 1000 learning_rate: 0.001 res: 61.500
num_iterations: 2000 learning_rate: 0.001 res: 61.500
num_iterations: 3000 learning_rate: 0.001 res: 60.000
num_iterations: 500 learning_rate: 0.005 res: 60.500
num_iterations: 1000 learning_rate: 0.005 res: 61.000
num_iterations: 2000 learning_rate: 0.005 res: 61.000
num_iterations: 3000 learning_rate: 0.005 res: 60.500
num_iterations: 500 learning_rate: 0.01 res: 57.000
num_iterations: 1000 learning_rate: 0.01 res: 59.000
num_iterations: 2000 learning_rate: 0.01 res: 59.500
num_iterations: 3000 learning_rate: 0.01 res: 60.000
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
```

```
num_iterations: 500 learning_rate: 0.05 res: 60.000
num_iterations: 1000 learning_rate: 0.05 res: 57.500
num_iterations: 2000 learning_rate: 0.05 res: 57.000
num_iterations: 3000 learning_rate: 0.05 res: 58.000
best score LogisticRegression 61.5
best num_iterations and learning_rate (1000, 0.001)
```

```
In [12]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat
```

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 0.525502
Cost after iteration 200: 1.079140
Cost after iteration 300: 0.784672
Cost after iteration 400: 0.751585
Cost after iteration 500: 0.685699
Cost after iteration 600: 0.583334
Cost after iteration 700: 0.529964
Cost after iteration 800: 0.539288
Cost after iteration 900: 0.559167
Training accuracy: 98.56459330143541 %
Test accuracy: 70.0 %
```

1.2.4 - Recherche aléatoire avec validation croisée

	Ensemble Total				
Itération 1	Test	Entraînement	Entraînement	Entraînement	Entraînement
Itération 2	Entraînement	Test	Entraînement	Entraînement	Entraînement
Itération 3	Entraînement	Entraînement	Test	Entraînement	Entraînement
Itération 4	Entraînement	Entraînement	Entraînement	Test	Entraînement
Itération 5	Entraînement	Entraînement	Entraînement	Entraînement	Test

Exercice 3 : Recherche aléatoire avec validation croisée

Complétez cette partie du code en vous basant sur le code ci-dessus.

```
In [13]: ## Recherche aléatoire avec validation croisée
# 16 points vérifiés
# 0.001 <= lr < 0.05
# 500 <= ni < 3000
# Validation croisée à 5 blocs (40 échantillons dans chaque bloc)

np.random.seed(1)
best_score=0
for iter in range(16):
    lr = 0.001 + (0.05-0.001) * np.random.rand()
    ni = np.random.randint(500,3000)
    cv_scores = []
    for cv in range(5):
        X = train_set_x
        y = train_set_y
        logistic_regression_model = model(np.hstack((X[:, :(cv*40)], X[:, (cv*40)+40:])), np.hstack((y[:, :(cv*40)], y[:, (cv*40)+40:])))
        cv_score = logistic_regression_model["res"]
        cv_scores.append(cv_score)
    mean_score=np.mean(cv_scores)
    print("num_iterations: ", ni, "learning_rate: ", lr, 'res: {:.3f}'.format(mean_score))
    if mean_score > best_score:
        best_score = mean_score
        best_score_combination = (ni, lr)
print("best score LogisticRegression",best_score)
print("best num_iterations and learning_rate",best_score_combination)
```

```
num_iterations: 1596 learning_rate: 0.02143407823042613 res: 58.000
num_iterations: 1460 learning_rate: 0.0010056043660498994 res: 61.500
num_iterations: 1702 learning_rate: 0.005524591143671092 res: 60.500
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
```

```
num_iterations: 857 learning_rate: 0.04246923491761484 res: 57.500
num_iterations: 752 learning_rate: 0.03457575551944122 res: 58.000
num_iterations: 2946 learning_rate: 0.04402775438315633 res: 59.000
num_iterations: 1253 learning_rate: 0.03385290799874171 res: 58.500
num_iterations: 2388 learning_rate: 0.007878959991166456 res: 60.000
num_iterations: 2525 learning_rate: 0.03914107258049014 res: 57.000
num_iterations: 819 learning_rate: 0.048444817210250485 res: 58.000
num_iterations: 1837 learning_rate: 0.005547239623396158 res: 61.000
num_iterations: 816 learning_rate: 0.04394306846250588 res: 57.000
num_iterations: 1228 learning_rate: 0.04162819846098994 res: 58.500
num_iterations: 931 learning_rate: 0.002913684378411236 res: 60.000
num_iterations: 1042 learning_rate: 0.00390291686377661 res: 60.500
num_iterations: 1978 learning_rate: 0.030060210396604788 res: 58.000
best score LogisticRegression 61.5
best num_iterations and learning_rate (1460, 0.0010056043660498994)
```

Output attendu :

```
best score LogisticRegression 62.0
best num_iterations and learning_rate (1460, 0.0010056043660498994)
```

II - Réalisation à l'aide de Scikit Learn

2.1 - Optimisation

2.1.1 - Recherche par grille

```
In [14]: from sklearn.base import BaseEstimator, ClassifierMixin

class ModelWrapper(BaseEstimator, ClassifierMixin):
    def __init__(self, num_iterations=2000, learning_rate=0.5):
        self.num_iterations = num_iterations
        self.learning_rate = learning_rate

    def fit(self, X, y):
        self.model = model(X.T, y.T, test_set_x, test_set_y, num_iterations=self.num_iterations)
        self.classes_ = np.unique(y)

    def predict(self, X):
        Y_prediction = predict(self.model["w"], self.model["b"], X.T)
        return Y_prediction.T

    def score(self, X, y):
        Y_prediction = self.predict(X)
        return 100 - np.mean(np.abs(Y_prediction - y)) * 100
```

```
In [15]: from sklearn.model_selection import GridSearchCV
```

```
param_grid = {  
    'num_iterations': [500, 1000, 2000, 3000],  
    'learning_rate': [0.001, 0.005, 0.01, 0.05]  
}  
  
grid_search = GridSearchCV(ModelWrapper(), param_grid, scoring='accuracy', cv=3)  
grid_search.fit(train_set_x.T, train_set_y.T.ravel())  
  
print("Grid Search best parameters:", grid_search.best_params_)  
print("Grid Search best score:", grid_search.best_score_)
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log  
    cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.  
sum(w**2)  
  
Grid Search best parameters: {'learning_rate': 0.001, 'num_iterations': 2000}  
Grid Search best score: 0.6170462387853691
```

```
In [16]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat:
```

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 0.592785
Cost after iteration 200: 0.489271
Cost after iteration 300: 0.414092
Cost after iteration 400: 0.386026
Cost after iteration 500: 0.375477
Cost after iteration 600: 0.370414
Cost after iteration 700: 0.369337
Cost after iteration 800: 0.371257
Cost after iteration 900: 0.375487
Training accuracy: 96.65071770334929 %
Test accuracy: 72.0 %
```

```
In [17]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat:
```

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 0.525502
Cost after iteration 200: 1.079140
Cost after iteration 300: 0.784672
Cost after iteration 400: 0.751585
Cost after iteration 500: 0.685699
Cost after iteration 600: 0.583334
Cost after iteration 700: 0.529964
Cost after iteration 800: 0.539288
Cost after iteration 900: 0.559167
Training accuracy: 98.56459330143541 %
Test accuracy: 70.0 %
```

```
In [18]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat:
```

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 0.592785
Cost after iteration 200: 0.489271
Cost after iteration 300: 0.414092
Cost after iteration 400: 0.386026
Training accuracy: 92.3444976076555 %
Test accuracy: 74.0 %
```

```
In [19]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat:
```

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 0.591905
Cost after iteration 200: 0.557646
Cost after iteration 300: 0.532427
Cost after iteration 400: 0.512206
Training accuracy: 77.51196172248804 %
Test accuracy: 56.0 %
```

2.1.2 - Recherche aléatoire

```
In [20]: from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    'num_iterations': [500, 1000, 2000, 3000, 4000, 5000],
    'learning_rate': [0.001, 0.005, 0.01, 0.05]
}

random_search = RandomizedSearchCV(ModelWrapper(), param_dist, scoring='accuracy', cv=3, n_iter=100)
random_search.fit(train_set_x.T, train_set_y.T.ravel())

print("Random Search best parameters:", random_search.best_params_)
print("Random Search best score:", random_search.best_score_)

/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)

Random Search best parameters: {'num_iterations': 500, 'learning_rate': 0.001}
Random Search best score: 0.6123533471359558
```

```
In [21]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat:
```

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 1.858750
Cost after iteration 200: 6.214113
Cost after iteration 300: 4.870427
Cost after iteration 400: 10.665259
Cost after iteration 500: 5.324578
Cost after iteration 600: 5.699910
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
```

```
cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
```

```
Cost after iteration 700: 5.738623
Cost after iteration 800: 5.709903
Cost after iteration 900: 5.684913
Cost after iteration 1000: 5.660457
Cost after iteration 1100: 5.635786
Cost after iteration 1200: 5.610705
Cost after iteration 1300: 5.585189
Cost after iteration 1400: 5.559273
Cost after iteration 1500: 5.533009
Cost after iteration 1600: 5.506454
Cost after iteration 1700: 5.479661
Cost after iteration 1800: 5.452680
Cost after iteration 1900: 5.425556
Cost after iteration 2000: 5.398330
Cost after iteration 2100: 5.371036
Cost after iteration 2200: 5.343709
Cost after iteration 2300: 5.316376
Cost after iteration 2400: 5.289063
Cost after iteration 2500: 5.261794
Cost after iteration 2600: 5.234591
Cost after iteration 2700: 5.207471
Cost after iteration 2800: 5.180453
Cost after iteration 2900: 5.153551
Cost after iteration 3000: 5.126781
Cost after iteration 3100: 5.100153
Cost after iteration 3200: 5.073681
Cost after iteration 3300: 5.047373
Cost after iteration 3400: 5.021241
Cost after iteration 3500: 4.995291
Cost after iteration 3600: 4.969532
Cost after iteration 3700: 4.943970
Cost after iteration 3800: 4.918611
Cost after iteration 3900: 4.893462
Cost after iteration 4000: 4.868526
Cost after iteration 4100: 4.843808
Cost after iteration 4200: 4.819312
Cost after iteration 4300: 4.795041
Cost after iteration 4400: 4.770998
Cost after iteration 4500: 4.747187
Cost after iteration 4600: 4.723609
Cost after iteration 4700: 4.700265
Cost after iteration 4800: 4.677159
Cost after iteration 4900: 4.654291
Training accuracy: 100.0 %
Test accuracy: 66.0 %
```

```
In [22]: logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat:
```

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 1.858750
Cost after iteration 200: 6.214113
Cost after iteration 300: 4.870427
Cost after iteration 400: 10.665259
Cost after iteration 500: 5.324578
Cost after iteration 600: 5.699910
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
```

```
Cost after iteration 700: 5.738623
Cost after iteration 800: 5.709903
Cost after iteration 900: 5.684913
Cost after iteration 1000: 5.660457
Cost after iteration 1100: 5.635786
Cost after iteration 1200: 5.610705
Cost after iteration 1300: 5.585189
Cost after iteration 1400: 5.559273
Cost after iteration 1500: 5.533009
Cost after iteration 1600: 5.506454
Cost after iteration 1700: 5.479661
Cost after iteration 1800: 5.452680
Cost after iteration 1900: 5.425556
Training accuracy: 100.0 %
Test accuracy: 66.0 %
```

2.1.3 - Optimisation bayésienne

In [23]: `from skopt import BayesSearchCV`

```
opt = BayesSearchCV(
    ModelWrapper(),
    {
        'num_iterations': (500, 5000),
        'learning_rate': (0.001, 0.05)
    },
    n_iter=10,
    scoring='accuracy',
    cv=3
)

opt.fit(train_set_x.T, train_set_y.T.ravel())

print("Bayesian Optimization best parameters:", opt.best_params_)
print("Bayesian Optimization best score:", opt.best_score_)
```

```
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)
/var/folders/ph/k5sv7srx4ss999mm54kt90k00000gn/T/ipykernel_35627/3386352034.py:10: RuntimeWarning: divide by zero encountered in log
  cost = -1/n * (np.dot(Y,np.log(hat_Y).T) + np.dot((1-Y),np.log(1 - hat_Y).T)) + lambd * np.sum(w**2)

Bayesian Optimization best parameters: OrderedDict([('learning_rate', 0.0015915796539502022), ('num_iterations', 1797)])
Bayesian Optimization best score: 0.6170462387853691
```

In [24]: `logistic_regression_model = model(train_set_x, train_set_y, test_set_x, test_set_y, num_iterat`

```
Cost after iteration 0: 0.693147
Cost after iteration 100: 6.187985
Cost after iteration 200: 1.277553
Cost after iteration 300: 5.576595
Cost after iteration 400: 1.753637
Cost after iteration 500: 2.220414
Cost after iteration 600: 1.771812
Cost after iteration 700: 1.723884
Cost after iteration 800: 1.729173
Cost after iteration 900: 1.747281
Cost after iteration 1000: 1.769297
Training accuracy: 99.52153110047847 %
Test accuracy: 68.0 %
```

2.2 - le surapprentissage

2.2.1 - LASSO et Ridge

```
In [25]: from sklearn.linear_model import Ridge

## Définir l'espace des paramètres
param_grid = {
    'alpha': [0.01, 0.1, 0.5, 1.0],
    'max_iter': [500, 1000, 2000, 3000]
}

## Créer un modèle Ridge
ridge_model = Ridge()

## Intégration de méthodes dans GridSearchCV
grid_search = GridSearchCV(ridge_model, param_grid, scoring='accuracy', cv=5, verbose=1, n_jobs=1)

## Effectuer une recherche dans la grille
grid_search.fit(train_set_x.T, train_set_y.T)

## Imprimer les meilleurs paramètres et le meilleur score
print("Best parameters:", grid_search.best_params_)
print("Best score:", grid_search.best_score_)

## Créer une instance de modèle Ridge en utilisant les meilleurs paramètres obtenus
best_alpha = grid_search.best_params_['alpha']
best_max_iter = grid_search.best_params_['max_iter']
ridge_model = Ridge(alpha=best_alpha, max_iter=best_max_iter)

## Modèle de formation
ridge_model.fit(train_set_x.T, train_set_y.T)

## Prédiction
Y_pred = ridge_model.predict(train_set_x.T)
100 - np.mean(np.abs(Y_pred - test_set_y)) * 100

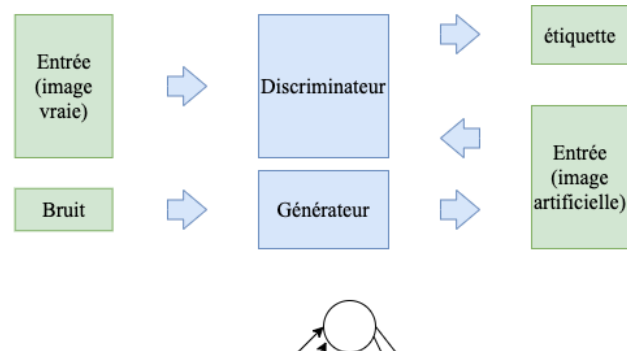
Fitting 5 folds for each of 16 candidates, totalling 80 fits
Best parameters: {'alpha': 0.01, 'max_iter': 500}
Best score: nan

/Library/Frameworks/Python.framework/Versions/3.8/lib/python3.8/site-packages/sklearn/model_selection/_search.py:979: UserWarning: One or more of the test scores are non-finite: [nan nan nan nan nan nan nan nan nan nan nan nan nan nan]
  warnings.warn(

Out[25]: 45.02375597714092
```

GAN

3 - Réseaux Adverses Génératifs (GAN)



3.1 - Générateur

Exercice 4 Implémentez la partie Générateur décrite dans la Figure.

Instructions:

1. Structure: Couche d'entrée → Couche cachée (activation tanh) → Couche de sortie (activation tanh)
2. `forward()` : Reçoit un bruit d'entrée z et génère un échantillon faux
3. `backward()` : Calcule les gradients (règle de la chaîne)
 - $dz2$: Gradient de la fonction de perte par rapport à la sortie du générateur
 - $dh2 = dz2 * (1 - h2^2)$: Dérivée de tanh
 - Rétropropagation du gradient couche par couche

```
In [27]: class Generator:
def __init__(self, input_dim, hidden_dim, output_dim):
    ##### 4 lignes #####
    self.W1 = np.random.randn(input_dim, hidden_dim) * 0.01
    self.b1 = np.zeros((1, hidden_dim))
    self.W2 = np.random.randn(hidden_dim, output_dim) * 0.01
    self.b2 = np.zeros((1, output_dim))
    #####

def forward(self, z):
    self.z = z # Sauvegarder les données d'entrée
    ##### 4 lignes #####
    self.z1 = np.dot(z, self.W1) + self.b1
    self.h1 = np.tanh(self.z1)
    self.z2 = np.dot(self.h1, self.W2) + self.b2
    self.h2 = np.tanh(self.z2)
    #####
    return self.h2

def backward(self, dz2):
    ##### 6 lignes #####
    dh2 = dz2 * (1 - self.h2 ** 2)
    dW2 = np.dot(self.h1.T, dh2)
    db2 = np.sum(dh2, axis=0, keepdims=True)
    dh1 = np.dot(dh2, self.W2.T) * (1 - self.h1 ** 2)
    dW1 = np.dot(self.z.T, dh1)
    db1 = np.sum(dh1, axis=0, keepdims=True)
    #####
    return dW1, db1, dW2, db2
```

Questions supplémentaires:

1. Pourquoi le générateur utilise-t-il `tanh` comme fonction d'activation de la couche de sortie? Quelle est sa plage de sortie?
2. Dans `backward()`, quelle est la base mathématique de la ligne `da1 = np.dot(da2, self.W2.T) * (1 - self.a1**2)` ?
3. Si on remplace la fonction d'activation de la couche cachée par ReLU, comment devrait-on modifier le code? Quels seraient les impacts?

Solutions:

1. La fonction `tanh` produit des sorties dans l'intervalle $[-1, 1]$. C'est utile pour normaliser les sorties du générateur. De plus, pour certains types de données (comme les images normalisées entre -1 et 1), cela correspond bien.
2. Cette ligne correspond à deux étapes:

- `np.dot(da2, self.W2.T)` : Propagation du gradient à travers la couche linéaire (de `a1` à `z2`)
- `(1 - self.a1**2)`: Multiplication par la dérivée de la fonction d'activation `tanh` (car `a1 = tanh(z1)`)

Ainsi, on calcule le gradient par rapport à `z1`

3. Pour utiliser ReLU:

```
self.a1 = np.maximum(0, self.z1)
```

```
da1 = np.dot(da2, self.W2.T) * (self.a1 > 0).astype(float)
```

Impacts: ReLU peut éviter le problème de gradient vanishing mais peut causer des neurones "morts" (dying ReLU) dans

3.2 - Discriminateur

Exercice 5 Implémentez la partie Discriminateur décrite dans la Figure.

Instructions:

1. Structure: Couche d'entrée → Couche cachée (`tanh`) → Couche de sortie (`sigmoid`)
2. `forward()` : Sort la probabilité que l'échantillon soit réel (entre 0 et 1)
3. `backward()` : Calcule le gradient de l'entropie croisée binaire
 - `dh2 = y_pred - y_true` : Propriété spéciale de la combinaison `sigmoid` + entropie croisée
 - Propagation du gradient similaire au générateur

```
In [34]: class Discriminator:
def __init__(self, input_dim, hidden_dim, output_dim):
    ##### 4 lignes #####
    self.W1 = np.random.randn(input_dim, hidden_dim) * 0.01
    self.b1 = np.zeros((1, hidden_dim))
    self.W2 = np.random.randn(hidden_dim, output_dim) * 0.01
    self.b2 = np.zeros((1, output_dim))
    #####

def forward(self, x):
    self.x = x # Sauvegarder les données d'entrée
    ##### 4 lignes #####
    self.z1 = np.dot(x, self.W1) + self.b1
    self.h1 = np.tanh(self.z1)
    self.z2 = np.dot(self.h1, self.W2) + self.b2
    self.h2 = 1 / (1 + np.exp(-self.z2))
    #####
    return self.h2

def backward(self, y_true, y_pred):
    ##### 6 lignes #####
    dh2 = y_pred - y_true
    dW2 = np.dot(self.h1.T, dh2)
    db2 = np.sum(dh2, axis=0, keepdims=True)
    dh1 = np.dot(dh2, self.W2.T) * (1 - self.h1 ** 2)
    dW1 = np.dot(self.x.T, dh1)
    db1 = np.sum(dh1, axis=0, keepdims=True)
    #####
    return dW1, db1, dW2, db2
```

Questions supplémentaires:

1. Pourquoi le discriminateur utilise-t-il une sigmoïde comme fonction d'activation de la couche de sortie?
2. Expliquez la dérivation mathématique de `dh2 = y_pred - y_true` (indice: perte d'entropie croisée + dérivée de `sigmoid`)
3. Si on n'utilise pas `sigmoid` au dernier étage du discriminateur, comment devrait-on ajuster la fonction de perte?

Solutions:

1. La sigmoïde est utilisée car le discriminateur effectue une tâche de classification binaire (vrai/faux) et la sigmoïde produit une probabilité dans $[0, 1]$ qui peut être interprétée comme la confiance que l'échantillon soit réel.
2. La perte d'entropie croisée pour un échantillon est:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Dérivée par rapport à $\hat{y}$:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}$$

Dérivée de la sigmoïde: $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$ Donc: $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} = \hat{y} - y$

3. On devrait utiliser une fonction de perte appropriée comme la perte hinge ou la perte least squares, ou implémenter

3.3 - Boucle d'entraînement du GAN

Ici, on étudie un cas pratique et fondamental des GAN : l'apprentissage d'une distribution de données unidimensionnelle simple. Plus spécifiquement, elle démontre comment un GAN peut apprendre à générer des données qui imitent une distribution gaussienne (normale) standard.

Exercice 6 Réaliser l'entraînement du GAN.

Instructions:

1. Entraînement du discriminateur:
 - Mélanger les échantillons réels (étiquette 1) et générés (étiquette 0)
 - Minimiser la perte d'entropie croisée: $-\mathbb{E}[\log(D(x)) + \log(1 - D(G(z)))]$
2. Entraînement du générateur:
 - Objectif: Maximiser $\log(D(G(z)))$
 - Calcul du gradient: $dz2 = -1/(D(G(z)))$ (dérivation dans les exercices)
3. Stratégie d'entraînement alterné

```

In [36]: def train_gan(generator, discriminator, num_epochs, batch_size, learning_rate):
    ## Distribution de données réelles  $N(0,1)$ 
    real_data = np.random.normal(0, 1, (1000, 1))

    for epoch in range(num_epochs):
        for _ in range(len(real_data) // batch_size):
            ## échantillons réels
            ##### 2 lignes #####
            real_samples = real_data[np.random.randint(0, len(real_data), batch_size)]
            real_labels = np.ones((batch_size, 1))
            #####

            # échantillons générés
            z = np.random.normal(0, 1, (batch_size, 1))
            ##### 2 lignes #####
            fake_samples = generator.forward(z)
            fake_labels = np.zeros((batch_size, 1))
            #####

            ## Combiner échantillons réels et générés
            ##### 2 lignes #####
            all_samples = np.vstack((real_samples, fake_samples))
            all_labels = np.vstack((real_labels, fake_labels))
            #####

            ## 1. Entraîner le discriminateur
            ##### 7~10 lignes #####
            d_preds = discriminator.forward(all_samples)
            d_loss = -np.mean(all_labels * np.log(d_preds + 1e-7) + (1 - all_labels) * np.log(1 - d_preds))
            dW1, db1, dW2, db2 = discriminator.backward(all_labels, d_preds)
            discriminator.W1 -= learning_rate * dW1
            discriminator.b1 -= learning_rate * db1
            discriminator.W2 -= learning_rate * dW2
            discriminator.b2 -= learning_rate * db2
            #####

            ## 2. Entraîner le générateur
            z = np.random.normal(0, 1, (batch_size, 1))
            ##### 3 lignes #####
            fake_samples = generator.forward(z)
            d_preds_fake = discriminator.forward(fake_samples)
            g_loss = -np.mean(np.log(d_preds_fake + 1e-7))
            #####

            ## Perte et gradient du générateur
            ##### 6~9 lignes #####
            dz2 = -1 / (d_preds_fake + 1e-7)
            dW1, db1, dW2, db2 = generator.backward(dz2)
            generator.W1 -= learning_rate * dW1
            generator.b1 -= learning_rate * db1
            generator.W2 -= learning_rate * dW2
            generator.b2 -= learning_rate * db2
            #####

            if epoch % 100 == 0:
                print(f"Epoch {epoch}, D Loss: {d_loss}, G Loss: {g_loss}")

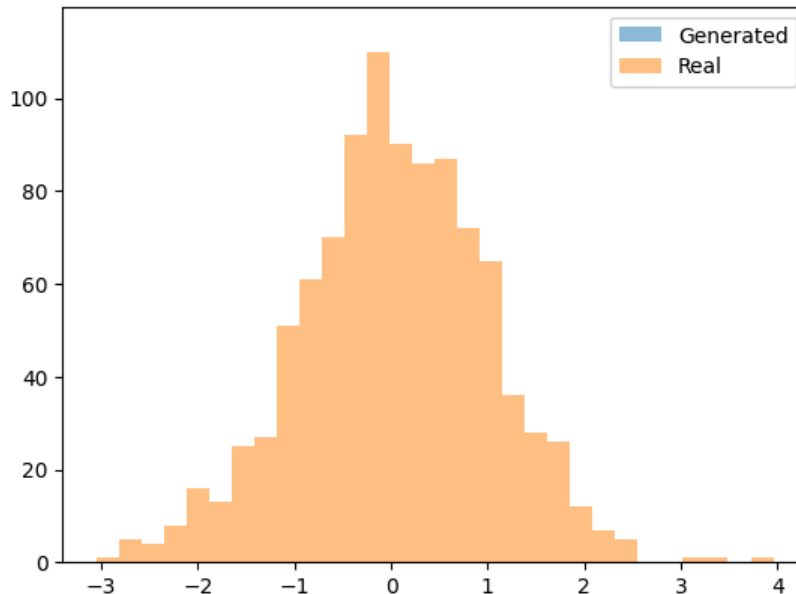
            # Échantillons générés vs distribution réelle
            z = np.random.normal(0, 1, (1000, 1))
            generated_samples = generator.forward(z)
            plt.hist(generated_samples, bins=30, alpha=0.5, label="Generated")
            plt.hist(real_data, bins=30, alpha=0.5, label="Real")
            plt.legend()
            plt.show()

        # Initialisation du réseau
        generator = Generator(input_dim=1, hidden_dim=10, output_dim=1)
        discriminator = Discriminator(input_dim=1, hidden_dim=10, output_dim=1)

        # Entraîner le GAN
        train_gan(generator, discriminator, num_epochs=1000, batch_size=32, learning_rate=0.01)

```

Epoch 0, D Loss: 0.6124650001113741, G Loss: 0.8899486701126634
 Epoch 100, D Loss: 0.11205522075044667, G Loss: 2.5606023808594687
 Epoch 200, D Loss: 0.10843898094730199, G Loss: 2.548602847328567
 Epoch 300, D Loss: 0.1042835253092943, G Loss: 2.6114662190026054
 Epoch 400, D Loss: 0.09984826504050234, G Loss: 2.7038884104589935
 Epoch 500, D Loss: 0.09674897469612488, G Loss: 2.7565260230349766
 Epoch 600, D Loss: 0.09476272787879902, G Loss: 2.7625271877410436
 Epoch 700, D Loss: 0.09279340285652249, G Loss: 2.7675680914529988
 Epoch 800, D Loss: 0.0908526728497397, G Loss: 2.770751280723915
 Epoch 900, D Loss: 0.08894425907386314, G Loss: 2.771570398420181



Questions supplémentaires:

1. Pourquoi ajoute-t-on $1e-7$ dans le calcul de la perte? Quel est le risque si on le supprime?
2. Dérivez mathématiquement le gradient du générateur $dz2 = -1/(d_preds_fake + 1e-7)$
3. Si le discriminateur est "trop bon" ($D(G(z)) \rightarrow 0$), quel est l'impact sur le gradient du générateur? Comment résoudre ce problème?
4. Dans le code, on entraîne d'abord le discriminateur puis le générateur. Que se passerait-il si on inversait l'ordre?

Solutions:

1. On ajoute $1e-7$ pour éviter le $\log(0)$ qui causerait une erreur numérique (NaN). Sans cela, si une prédiction est exactement 0 ou 1, le logarithme serait infini.
2. La perte du générateur est:

$$L_G = -\log(D(G(z)))$$

Dérivée par rapport à $D(G(z))$:

$$\frac{\partial L_G}{\partial D(G(z))} = -\frac{1}{D(G(z))}$$

Donc $dz2$ est cette dérivée, qui sera propagée à travers le discriminateur.

3. Le gradient devient très grand (explosion de gradient), rendant l'entraînement instable. Solutions:

- Utiliser la perte alternative $L_G = -\log(1 - D(G(z)))$
 - Implémenter la perte de Wasserstein
 - Réduire le taux d'apprentissage
4. Le générateur serait entraîné avec un discriminateur "périmé", ce qui pourrait ralentir la convergence. Mais dans la pratique, l'ordre peut être inversé sans problème majeur.

Avec ce cas-là:

1. **Illustration du principe fondamental** : Même avec une architecture minimaliste, le GAN capture l'essence de l'apprentissage génératif
2. **Diagnostic facile** : Les problèmes typiques des GAN (mode collapse, instabilité) sont immédiatement visibles :
 - Si l'histogramme généré ne ressemble pas à une gaussienne
 - Si les pertes divergent au lieu de converger vers 0.693
3. **Base pour des extensions** : Comprendre ce cas permet d'aborder des applications complexes (Extension potentielle à MNIST) :

```
generator = Generator(input_dim=100, hidden_dim=256, output_dim=784) # 28x28 images
```

```
discriminator = Discriminator(input_dim=784, hidden_dim=256, output_dim=1)
```

Questions générales::

1. Si les échantillons générés et la distribution réelle ne se chevauchent pas du tout, quelles en sont les causes possibles?
2. Comment modifier le code pour implémenter un GAN conditionnel (générer des données d'une catégorie spécifique)?

Solutions:

1. Causes possibles:
 - Effondrement de mode (mode collapse)
 - Taux d'apprentissage inadapté
 - Architecture réseau trop simple
 - Équilibre incorrect entre l'entraînement du générateur et du discriminateur
2. Pour un GAN conditionnel:
 - Ajouter des étiquettes aux entrées du générateur et du discriminateur
 - Concaténer le bruit d'entrée avec l'étiquette conditionnelle
 - Augmenter la dimension des couches pour accommoder les entrées supplémentaires

In []: